Project Title

Mini Search Engine/ Information-Retrieval System

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Uppu Sindhuja (2520080045)
P. Bhagya Sri (2520080024)
Santosh (2520090051)

Team Number: 13

Section: 13

Under the guidance of

Dr. Dyuti Benerjee

Asst. Professor, AI&DS

(DEEMED TO BE UNIVERSITY)
BACHUPALLY
Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Mini Search Engine/ Information-Retrieval System"** is a record of Bonafide work of **U. Sindhuja–2520080045, P. Bhagya Sri– 2520080024, Santosh–2520090051.** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artificial Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

U. Sindhuja– 2520080045.

P. Bhagya Sri – 2520080024

Santosh – 2520090051.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Mini Search Engine/ Information-Retrieval System"** is a bonafide work done and submitted by **U. Sindhuja–2520080045, P. Bhagya Sri– 2520080024, Santosh–2520090051.** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, <<Guide Name>>, <<Designation>>, <Department>> which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

U. Sindhuja– 2520080045.

P. Bhagya Sri – 2520080024.

Santosh - 2520090051.

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

The Mini Search Engine, also known as an Information Retrieval System, is a document retrieval application designed to search Microsoft Word documents and retrieve relevant information efficiently. Its objective is to deliver accurate and useful results by applying fundamental Information Retrieval concepts and Data Structures and Algorithms techniques. Instead of scanning every file for each query, the application preprocesses document text and builds an inverted index that maps terms to the documents in which they appear, enabling faster keyword lookup. Users can upload multiple documents, view saved files, download originals, and delete documents when they are no longer needed. Files remain stored locally until deleted.

The system offers four search methods: keyword, exact phrase, spelling-tolerant fuzzy, and related-document search. Keyword queries use the inverted index, while phrase search uses the Knuth–Morris–Pratt algorithm to locate an exact sequence of words. Fuzzy search applies edit distance to suggest likely corrections for misspellings. Related-document search represents documents as term-frequency vectors and uses cosine similarity to rank documents with similar content. By implementing these core search algorithms directly, the project demonstrates text preprocessing, indexing, query processing, ranking, and similarity analysis, while enabling discussion of algorithmic time and space complexity and scalability trade-offs.

Team Name: Team 13
Team Logo (if any):



Team Members:

1. U. Sindhuja - 2520080045

2. P. Bhagya Sri - 2520080024
3. Santosh - 2520090051

2. INTRODUCTION

Introduction

As digital information grows, people often accumulate large collections of documents that become difficult to organize and search. Finding a particular topic or phrase by opening files one at a time can take considerable effort. WordVault, a Mini Search Engine and Information Retrieval System, addresses this problem by allowing users to store Microsoft Word .doc and .docx files locally and search their contents through a single website.

The project applies concepts from Information Retrieval and Data Structures and Algorithms to make document discovery more efficient. When a Word file is uploaded, the system extracts its text and creates an inverted index that maps each term to the documents in which it occurs. Instead of checking every document from the beginning for every query, the search process uses this index to locate relevant files. Users can choose from keyword search, exact-phrase search, spelling assistance, and related-document search. These methods demonstrate techniques including inverted indexing, Knuth–Morris–Pratt string matching, edit distance, and cosine similarity over term-frequency vectors.

WordVault also provides a dashboard for viewing the document collection, uploading multiple files, filtering the library by filename, downloading originals, and deleting files when they are no longer needed. The original documents remain stored on the user's computer between application sessions. Text extraction is supported by Apache POI, while the core indexing and search methods are implemented directly to make their logic understandable.

This report describes the project's objectives and requirements, system design, data structures, search algorithms, implementation, and evaluation. It also discusses the

algorithms' time and space complexity, current limitations, and possible improvements.

3. LITERATURE SURVEY

3.1 Google Drive

Google Drive provides cloud-based storage with a search feature for locating files by name and content. Users can narrow results using filters such as file type, owner, and modification date. This makes it easier to find a document within a large collection without opening files one by one. WordVault similarly aims to make document retrieval easier, while focusing on locally stored Word documents and demonstrating search algorithms. Google Drive Help

3.2 Microsoft SharePoint

Microsoft SharePoint includes search across files and other stored content. Users can search with keywords or exact phrases and refine results using properties such as file type. Search results are ordered by relevance to help users find useful documents sooner. WordVault also offers keyword and phrase search, with a simpler interface designed for this project. Microsoft Support: Search for content

3.3 DocFetcher

DocFetcher is a desktop application for searching the contents of local documents. It supports Microsoft Word formats, including .doc and .docx, and offers phrase and fuzzy searches. Users can preview matching documents and see where the search terms appear. DocFetcher is closely related to WordVault because both focus on finding information in locally stored files. DocFetcher

3.4 Recoll

Recoll is a desktop full-text search tool that indexes files on a user's computer. It supports many document formats, including Word files, and can search document contents as well as file names. Search results can be previewed or opened from the application. WordVault follows a similar local-search approach, with a narrower focus on Word documents and a small set of learning-oriented search methods. Recoll

3.5 Paperless-ngx

Paperless-ngx is a web-based document management system that organizes documents in a searchable archive. It provides a web interface and lets users narrow documents using metadata such as tags and document types. Its search features help users retrieve stored documents from a growing collection. WordVault shares the goal of searchable document storage, while concentrating on Word files and core search algorithms. Paperless-ngx documentation

3.6 Apache Solr

Apache Solr is an open-source search platform that provides full-text search through an API. Its features include phrase matching, relevance-based results, spelling suggestions, and support for parsing rich document formats. Solr is built for flexible search applications and larger indexing needs. WordVault demonstrates some related search ideas in a smaller application without relying on a search-engine library. Apache Solr features

3.7 Elasticsearch

Elasticsearch is a search and analytics engine that uses an inverted index to retrieve documents efficiently. It analyzes text and uses relevance scoring to rank results for full-text queries. These indexing and ranking ideas are relevant to WordVault's search functionality. WordVault applies a simpler, project-scale approach to indexing and supports keyword, phrase, spelling, and document-similarity searches. Elasticsearch: How full-text search works

3.8 Summary

The reviewed systems show several approaches to document retrieval, including cloud search, local desktop search, document management, and dedicated search platforms. Common features include indexing, full-text or phrase search, filtering, previews, and relevance ranking. WordVault brings selected ideas together in a focused application for storing and searching Word documents. Its purpose is to demonstrate these retrieval methods through a straightforward user interface and project-level algorithms.

References

1. Google Drive Help, “Search for files in Google Drive.” https://support.google.com/drive/answer/2375114?hl=en-SG&ref_topic=14950478
2. Microsoft Support, “Search for content.” https://support.microsoft.com/en-us/sharepoint/search-for-content-1
3. DocFetcher. https://docfetcher.sourceforge.io/
4. Recoll. https://recoll.org/
5. Paperless-ngx documentation, “Usage.” https://docs.paperless-ngx.com/usage/
6. Apache Solr, “Features.” https://solr.apache.org/features
7. Elasticsearch documentation, “How full-text search works.” https://www.elastic.co/docs/solutions/search/full-text/how-full-text-works

4. Problem Statement and Objectives

Problem Statement:

Finding a specific Word document in a growing collection is difficult and time-consuming when files are stored without efficient indexing and search options.

Problem Description:

Users may need to open and check multiple files to find a document containing particular information. The project addresses this challenge by providing a central place to store Word documents and search their contents using different methods.

Objectives:

- Develop a website for uploading, viewing, downloading, and deleting Word documents.
- Store uploaded documents locally so they remain available until the user deletes them.
- Build an index of document contents to support faster searching.
- Provide keyword and exact-phrase search to locate relevant documents.
- Include spelling suggestions to help with misspelled search terms.
- Identify related documents based on similarities in their content.
- Present search and document-management features through a clear, easy-to-use interface.
- Demonstrate how information-retrieval and data-structure algorithms can be used in a practical application

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- **Processor:** Intel Core i3 or equivalent and above
- **Memory:** Minimum 4 GB RAM
- **Storage:** At least 1 GB of free disk space for the application and documents

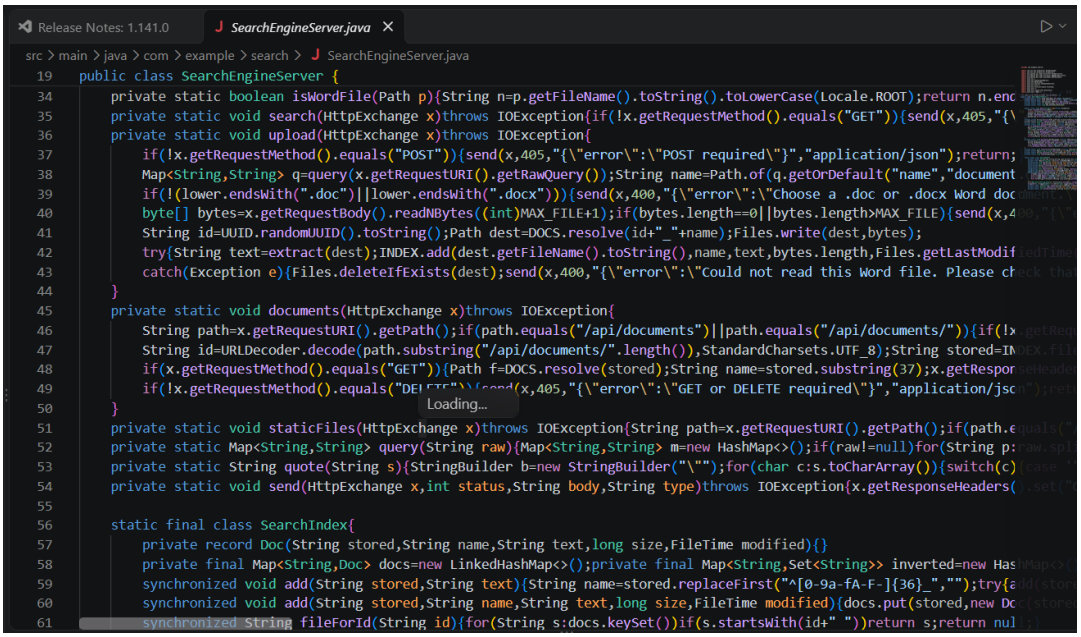
Software Requirements:

- **Operating System:** Windows 10 or later
- **Programming Language:** Java Development Kit (JDK) 17 or later
- **Build Tool:** Apache Maven
- **Browser:** Google Chrome, Microsoft Edge, or another modern web browser
- **Document Library:** Apache POI for reading .doc and .docx files

6. IMPLEMENTATION

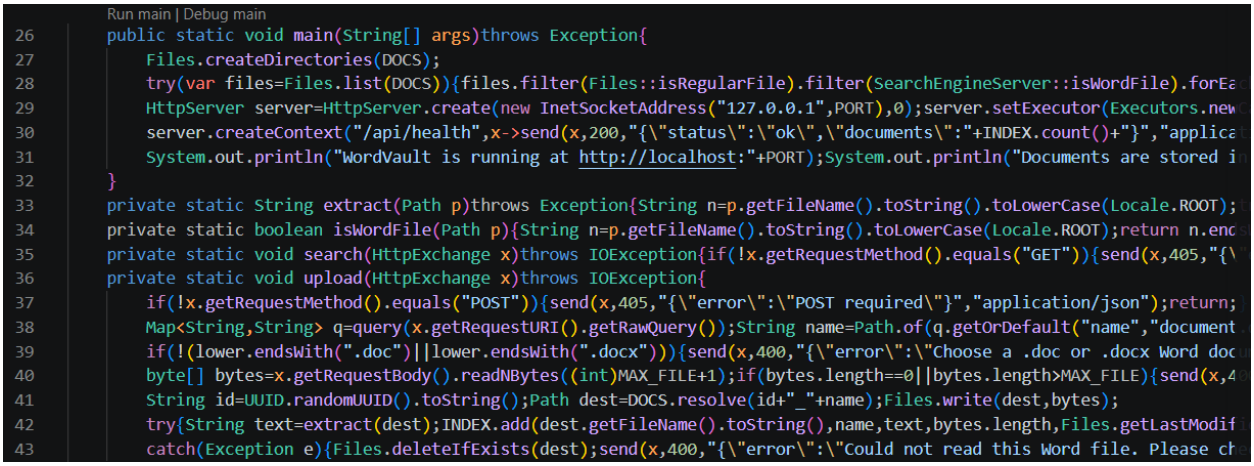
1. Word document upload and storage

This module accepts .doc and .docx files, checks their size, saves them in the project's document-storage folder, and indexes the extracted text. If a file cannot be read, the saved copy is removed.



2. Text extraction and inverted-index creation

Apache POI extracts text from Word files. The indexing code tokenizes that text, counts each word, and records which documents contain each word. This lets the search feature find matching documents through the index.



```
Run main | Debug main
    public static void main(String[] args)throws Exception{
        Files.createDirectories(DOCS);
        try(var files=Files.list(DOCS)){files.filter(Files::isRegularFile).filter(SearchEngineServer::isWordFile).forEa
        HttpServer server=HttpServer.create(new InetSocketAddress("127.0.0.1",PORT),0);server.setExecutor(Executors.new
        server.createContext("/api/health",x->send(x,200,"{\"status\":\"ok\",\"documents\":"+INDEX.count()+"}","applica
        System.out.println("WordVault is running at http://localhost:"+PORT);System.out.println("Documents are stored i
    }
    private static String extract(Path p)throws Exception{String n=p.getFileName().toString().toLowerCase(Locale.ROOT);
    private static boolean isWordFile(Path p){String n=p.getFileName().toString().toLowerCase(Locale.ROOT);return n.enc
    private static void search(HttpExchange x)throws IOException{if(!x.getRequestMethod().equals("GET")){send(x,405,"{\
    private static void upload(HttpExchange x)throws IOException{
        if(!x.getRequestMethod().equals("POST")){send(x,405,"{\"error\":\"POST required\"}","application/json");return;
        Map<String,String> q=query(x.getRequestURI().getRawQuery());String name=Path.of(q.getOrDefault("name","document
        if(!(lower.endsWith(".doc")||lower.endsWith(".docx"))){send(x,400,"{\"error\":\"Choose a .doc or .docx Word doc
        byte[] bytes=x.getRequestBody().readNBytes((int)MAX_FILE+1);if(bytes.length==0||bytes.length>MAX_FILE){send(x,4
        String id=UUID.randomUUID().toString();Path dest=DOCS.resolve(id+"_"+name);Files.write(dest,bytes);
        try{String text=extract(dest);INDEX.add(dest.getFileName().toString(),name,text,bytes.length,Files.getLastModif
        catch(Exception e){Files.deleteIfExists(dest);send(x,400,"{\"error\":\"Could not read this Word file. Please ch
```

3. Search and result ranking

The search module processes a query using the selected search mode. Keyword searches look up query terms in the inverted index, while phrase, spelling, and related-document searches use their corresponding algorithms. Matching documents are sorted by their scores and returned with a text snippet.



```
    static final class SearchIndex{
        private record Doc(String stored,String name,String text,long size,FileTime modified){}
        private final Map<String,Doc> docs=new LinkedHashMap<>();private final Map<String,Set<String>> inverted=new HashMap<>();private final Map<String,Map
        synchronized void add(String stored,String text){String name=stored.replaceFirst("^[0-9a-fA-F-]{36}_","");try{add(stored,name,text,Files.size(DOCS.r
        synchronized void add(String stored,String name,String text,long size,FileTime modified){docs.put(stored,new Doc(stored,name,text,size,modified));Ma
        synchronized String fileForId(String id){for(String s:docs.keySet())if(s.startsWith(id+"_"))return s;return null;}
        synchronized int count(){return docs.size();}
        synchronized void remove(String stored){Doc d=docs.remove(stored);freq.remove(stored);if(d!=null)for(String w:tokens(d.text())){Set<String> names=in
        synchronized String documentsJson(){StringBuilder b=new StringBuilder("[");boolean first=true;for(Doc d:docs.values()){if(!first)b.append(',');first
        synchronized String search(String raw,String mode){List<String> terms=tokens(raw);Map<String,Double> scores=new HashMap<>();if(!terms.isEmpty()){
            if(mode.equals("phrase")){String phrase=raw.toLowerCase(Locale.ROOT).trim().replaceAll("\\s+"," ");for(Doc d:docs.values())if(kmpContains(d.text
            else if(mode.equals("related")){Set<String> seeds=new HashSet<>();for(String t:terms)seeds.addAll(inverted.getOrDefault(t,Set.of()));Map<String,
            else if(mode.equals("spelling")){for(String t:terms){Set<String> files=inverted.get(t);String lookup=t;if(files==null||files.isEmpty()){int best
            else for(String t:terms)for(String file:inverted.getOrDefault(t,Set.of()))scores.merge(file,1+Math.log1p(freq.get(file).getOrDefault(t,0)),Doubl
        }
        List<Map.Entry<String,Double>> hits=scores.entrySet().stream().sorted(Map.Entry.<String,Double>comparingByValue().reversed().thenComparing(Map.Entry
        b.append("],\"suggestions\":[");Set<String> suggested=new TreeSet<>();if(!terms.isEmpty()&&(hits.isEmpty()||mode.equals("spelling")))for(String t:te
        private static List<String> tokens(String s){Matcher m=Pattern.compile("[\\p{L}\\p{N}]+").matcher(s.toLowerCase(Locale.ROOT));List<String> a=new Arr
        private static boolean kmpContains(String text,String pattern){if(pattern.isEmpty())return true;int[] lps=new int[pattern.length()];for(int i=1,len=
        private static int distance(String a,String b){int[] prev=new int[b.length()+1];for(int j=0;j<=b.length();j++)prev[j]=j;for(int i=1;i<=a.length();i+
    }
}
```

4. Document listing, download, and deletion

This module lists the stored documents, lets users download the original files, and deletes a document from both storage and the search index. This supports managing the document collection after upload.

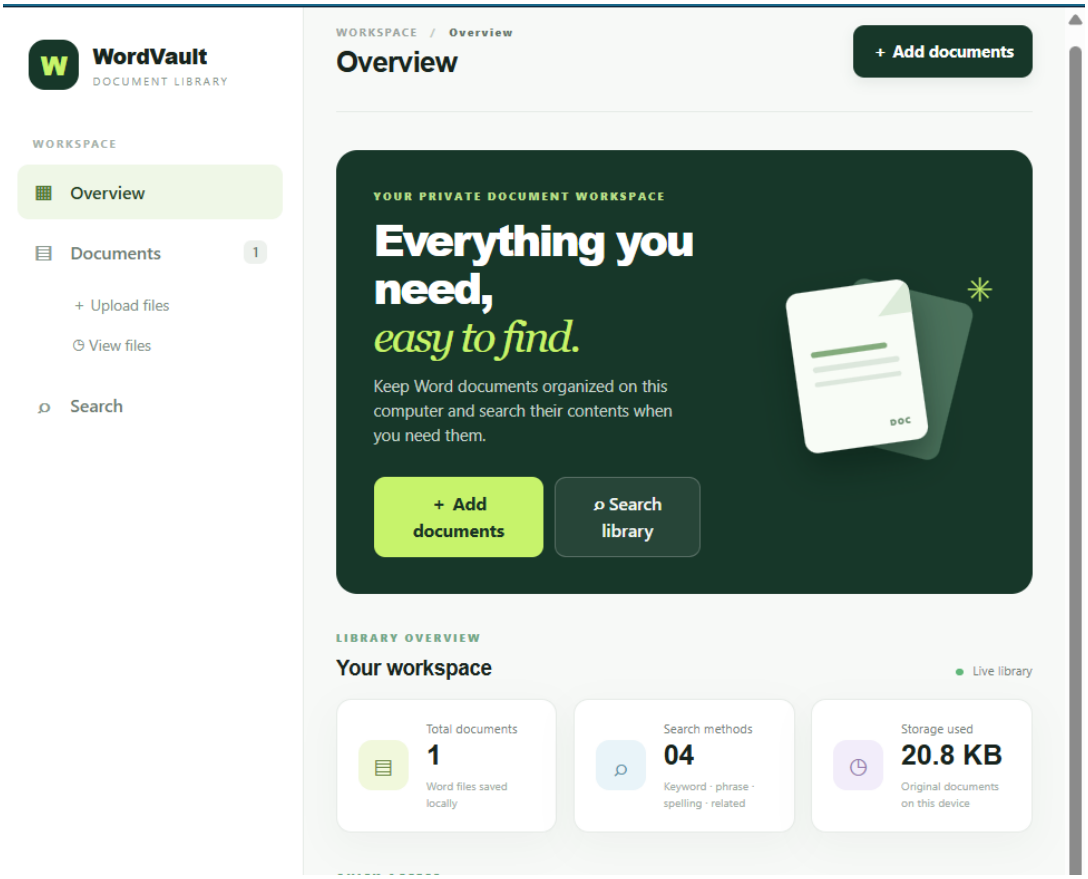
```
    private static void documents(HttpExchange x)throws IOException{
        String path=x.getRequestURI().getPath();if(path.equals("/api/documents")||path.equals("/api/documents/")){if(!x.getRequestMethod().equals("GET")){se
        String id=URLDecoder.decode(path.substring("/api/documents/".length()),StandardCharsets.UTF_8);String stored=INDEX.fileForId(id);if(stored==null){se
        if(x.getRequestMethod().equals("GET")){Path f=DOCS.resolve(stored);String name=stored.substring(37);x.getResponseHeaders().set("Content-Type","appli
        if(!x.getRequestMethod().equals("DELETE")){send(x,405,"{\"error\":\"GET or DELETE required\"}","application/json");return;}Files.deleteIfExists(DOCS
    }
    private static void staticFiles(HttpExchange x)throws IOException{String path=x.getRequestURI().getPath();if(path.equals("/"))path="/index.html";Path f=
    private static Map<String,String> query(String raw){Map<String,String> m=new HashMap<>();if(raw!=null)for(String p:raw.split("&")){String[] a=p.split("=
    private static String quote(String s){StringBuilder b=new StringBuilder("\"");for(char c:s.toCharArray()){switch(c){case '"'->b.append("\\\"");case '\\'
    private static void send(HttpExchange x,int status,String body,String type)throws IOException{x.getResponseHeaders().set("Content-Type",type);byte[] b=b

    static final class SearchIndex{
        private record Doc(String stored,String name,String text,long size,FileTime modified){}
        private final Map<String,Doc> docs=new LinkedHashMap<>();private final Map<String,Set<String>> inverted=new HashMap<>();private final Map<String,Map
        synchronized void add(String stored,String text){String name=stored.replaceFirst("^[0-9a-fA-F-]{36}_","");try{add(stored,name,text,Files.size(DOCS.r
        synchronized void add(String stored,String name,String text,long size,FileTime modified){docs.put(stored,new Doc(stored,name,text,size,modified));Ma
        synchronized String fileForId(String id){for(String s:docs.keySet())if(s.startsWith(id+"_"))return s;return null;}
        synchronized int count(){return docs.size();}
        synchronized void remove(String stored){Doc d=docs.remove(stored);freq.remove(stored);if(d!=null)for(String w:tokens(d.text())){Set<String> names=in
        synchronized String documentsJson(){StringBuilder b=new StringBuilder("[");boolean first=true;for(Doc d:docs.values()){if(!first)b.append(',');first
        synchronized String search(String raw,String mode){List<String> terms=tokens(raw);Map<String,Double> scores=new HashMap<>();if(!terms.isEmpty()){
```

7. RESULTS & DISCUSSION

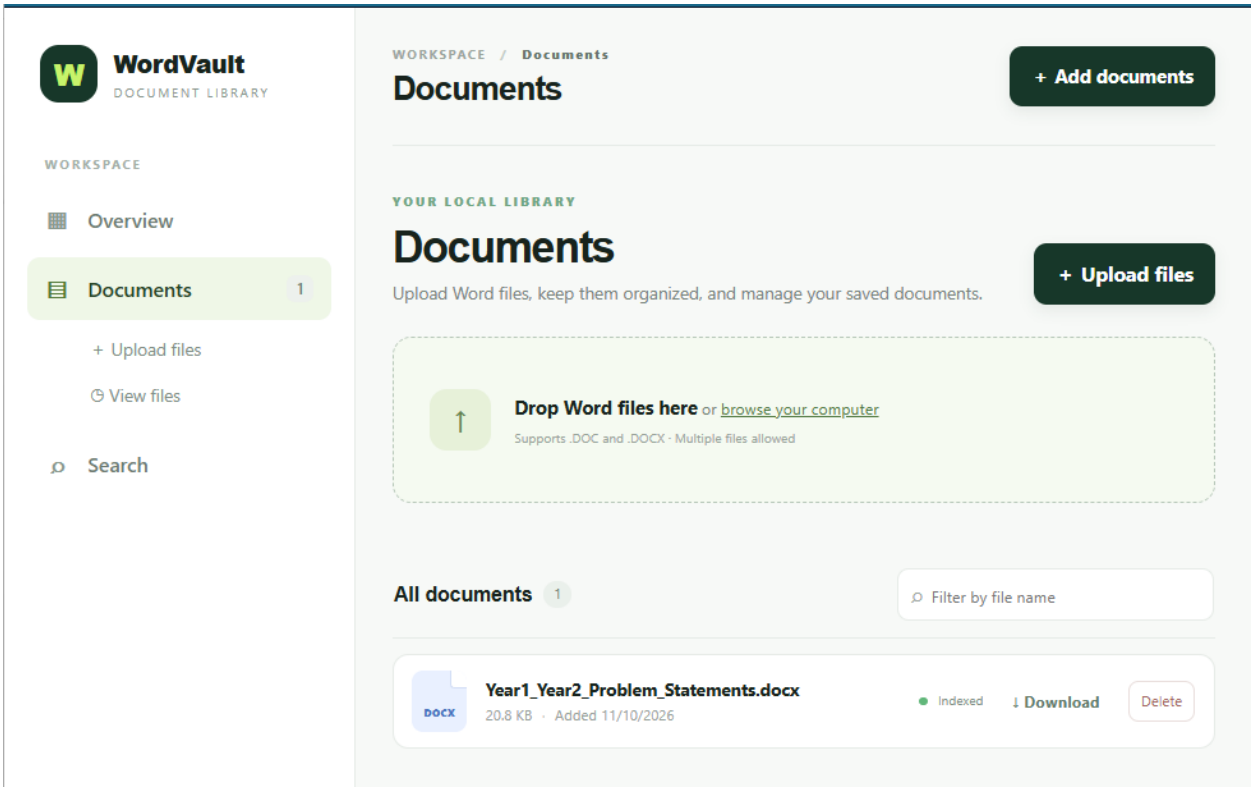
```
PS C:\Users\sindh\OneDrive\Desktop\WordVault-Project> mvn compile exec:java
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example:mini-search-engine >-------------------
[INFO] Building mini-search-engine 2.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ mini-search-engine ---
[INFO] Copying 3 resources from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ mini-search-engine ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- exec:3.3.0:java (default-cli) @ mini-search-engine ---
2026-10-11T16:56:09.895737900Z com.example.search.SearchEngineServer.main() ERROR Log4j API could not find a logging provider.
WordVault is running at http://localhost:8080
Documents are stored in: C:\Users\sindh\OneDrive\Desktop\WordVault-Project\data\documents
```

Figure 1: WordVault Dashboard



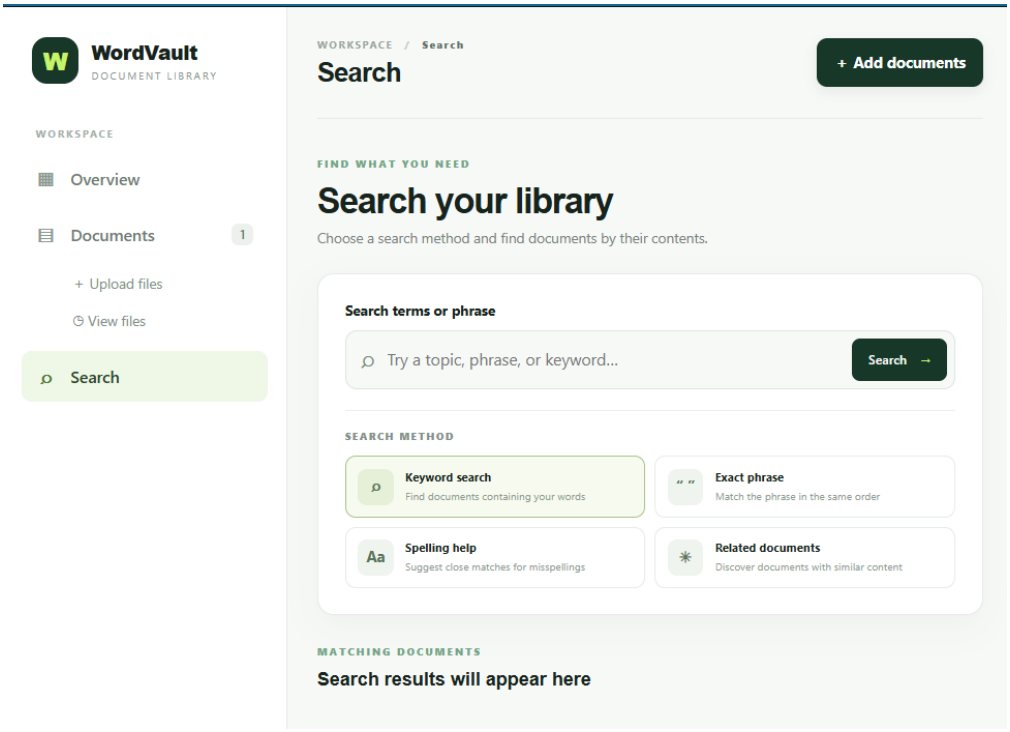
The dashboard displays an overview of the document library, including the number of stored documents and storage used. It also provides navigation to the Documents and Search sections.

Figure 2: Document Upload and Library



Users can upload .doc and .docx files, which are saved on the computer and added to the searchable library. The library provides options to download or delete stored documents.

Figure 3: Keyword Search Results



WordVault searches the document index for the entered term and displays matching documents with relevance scores and text previews. The results help users identify the document they need without opening every file.

The application combines document storage, indexing, and multiple search methods in one website. Search results depend on the Word documents uploaded to the library, so upload sample documents before taking the screenshots.

8. CONCLUSION & FUTURE SCOPE

Conclusion

WordVault was developed to make storing and finding Word documents easier. It allows users to upload, view, download, and delete .doc and .docx files, while keeping uploaded files on the local computer. The project also provides keyword search, exact-phrase search, spelling suggestions, and related-document search. These features meet the main objectives of the project and demonstrate how indexing and search algorithms can support document retrieval through a simple website.

Future Scope

Near-term enhancements

- Add folders, tags, and document categories to organize the library.
- Improve search results with highlighted matching words and more informative previews.
- Add support for additional document formats, such as PDF, if required.

Medium-term enhancements

- Add user accounts and permissions so multiple users can maintain separate libraries.
- Provide configurable backup and restore options for stored documents.
- Improve indexing and search performance for larger document collections.

Long-term possibilities

- Explore optical character recognition (OCR) to make scanned documents searchable.
- Study semantic search to find documents based on meaning, not only matching words.
- Evaluate deployment on a server or private cloud for access across multiple devices.

9. REFERENCES

1. **Google Drive Help, "Search for files in Google Drive." https://support.google.com/drive/answer/2375114?hl=en-SG&ref_topic=14950478**
2. **Microsoft Support, "Search for content." https://support.microsoft.com/en-us/sharepoint/search-for-content-1**
3. **DocFetcher, desktop document search application. https://docfetcher.sourceforge.io/**
4. **Recoll, full-text search application. https://recoll.org/**
5. **Paperless-ngx, document management system documentation. https://docs.paperless-ngx.com/usage/**
6. **Apache Solr, search platform features. https://solr.apache.org/features**
7. **Elasticsearch, "How full-text search works." https://www.elastic.co/docs/solutions/search/full-text/how-full-text-works**
8. **Apache POI, "HWPF and XWPF: Java API to Handle Microsoft Word Files." https://poi.apache.org/components/document/index**
9. **Apache Maven, official documentation. https://maven.apache.org/guides/**
10. **Oracle, JDK 17 documentation. https://docs.oracle.com/en/java/javase/17/**